

Theoretical Documentation: 3D Continuum Topology Optimization Engine with Linearized Buckling Stability

Author: Computational Engineering Multi-Agent Team (Gabriele Ricatto)

Date: October 2026

Document Class: Academic Theoretical Specification and Mathematical Log

Reference Document: T0 (1).pdf (“Multi-agentic implementation of a 2D topology optimization framework for compliance and linearized buckling via weighted sum method”, G. Ricatto, Sept 2026)

Abstract

This document presents the complete mathematical formulation and theoretical foundations of a three-dimensional (3D) continuum topology optimization framework designed to systematically explore the Pareto-optimal trade-off between structural compliance (global elastic stiffness) and the fundamental linearized Buckling Load Factor (BLF). Discretizing the physical domain via an 8-node trilinear hexahedral isoparametric finite element formulation (H8) over a structured Cartesian voxel grid, the structural elasticity and initial-stress geometric stiffness are formulated from the weak form of linear and geometrically non-linear continuum mechanics.

Material distribution is parameterized through the Solid Isotropic Material with Penalization (SIMP) interpolation scheme, mathematically grounded in the 3D Hashin-Shtrikman composite bounds ($p_K = 3.0$). To predict compressive instability while preventing computational bottlenecks, the 3D geometric stiffness matrix is synthesized by decomposing the Cauchy stress tensor into six constant invariant 24×24 basis matrices, enabling vectorized assembly without numerical quadrature during optimization iterations. Spurious localized buckling modes in void regions are rigorously suppressed via differential penalization ($p_G \geq p_K$). The multi-objective optimization problem is scalarized through an adaptive weighted sum method governed by dynamic L_1 -norm gradient normalization factors ($S_C^{(i)}, S_\mu^{(i)}$) and scalarization parameter $\alpha \in [0, 1]$, with Pareto efficiency guaranteed under Geoffrion’s theorem. Exact analytical sensitivities are derived via the adjoint state method requiring only a single back-solve using the pre-factored stiffness matrix. Regularization is enforced via a 3D spherical cone filter ($r_{\min}$) combined with a smoothed Heaviside projection continuation scheme (β -continuation), and the density field is iteratively updated through an unconditionally stable Optimality Criteria (OC) bisection scheme. Finally, the numerical architecture is tailored for extreme memory constraints (< 2 GB peak RAM on 8GB hardware) and seamlessly interfaced with an OpenVDB / PicoGK implicit geometry pipeline for watertight additive manufacturing.

1. Introduction and Architectural Paradigm

1.1 Continuum Topology Optimization vs. Generative Design

A fundamental methodological distinction exists between **Topology Optimization (TO)** and the broader engineering framework of **Generative Design (GD)**:

1. **Topology Optimization (TO)** originates from the variational calculus and mathematical mechanics communities [2, 6]. It is a deterministic, gradient-based numerical solver. Operating on a fixed design domain discretized by finite elements, TO formulates a mathematically bounded optimization problem: minimizing or maximizing an objective functional (such as structural compliance, natural frequency, or critical buckling load) subject to inequality constraints (such as volume fraction) through the continuous redistribution of a fictitious material density field $\mathbf{x} \in [0, 1]^m$. TO yields an abstract spatial density distribution.
2. **Generative Design (GD)** represents a holistic, multi-disciplinary engineering paradigm spanning the entire trajectory from functional specification to physical manufacturing. Within GD, Topology Optimization serves as the **core physics and form-generation engine**. However, GD encompasses:

- Automated functional requirement formulation (prescribed loads, support degrees of freedom, keep-out passive void exclusion zones, and keep-in mounting interfaces).
- Multi-physics simulation (linear static equilibrium, thermal diffusion, and linearized stability analysis).
- Manufacturing process constraint integration (additive manufacturing self-supporting overhang angles, casting draft directions, and minimum feature wall thicknesses).
- Implicit geometry synthesis: transmuting raw voxel densities into smooth, manifold, watertight boundary representations with continuous fillets—achieved in this project via **PicoGK** and OpenVDB lattice ray-marching.

1.2 Mathematical Foundations of Topology Invariance under Load Reversal

In conventional minimum compliance topology optimization governed purely by linear elasticity, applying an external force $\mathbf{F}$ or its exact inverse $-\mathbf{F}$ produces an **identical optimized topology**. This invariance is a direct mathematical consequence of linear elastostatics.

Static equilibrium is governed by Hooke’s law:

$$\mathbf{K}(\rho)\mathbf{U} = \mathbf{F} \quad (1)$$

where $\mathbf{K}$ is the symmetric, positive-definite global stiffness matrix, $\mathbf{U}$ is the nodal displacement vector, and $\mathbf{F}$ is the applied external force vector. Inverting the load vector to $-\mathbf{F}$ yields an identically reversed displacement field:

$$\mathbf{U}_{-\mathbf{F}} = \mathbf{K}^{-1}(-\mathbf{F}) = -\mathbf{U} \quad (2)$$

Structural compliance C represents the total external work done by the loads, which equals twice the internal elastic strain energy:

$$C = \mathbf{F}^T \mathbf{U} = \mathbf{U}^T \mathbf{K} \mathbf{U} = \sum_{e=1}^m E(\rho_e) \mathbf{u}_e^T \mathbf{k}_0 \mathbf{u}_e \quad (3)$$

Evaluating the compliance under the reversed load $-\mathbf{F}$ gives:

$$C_{-\mathbf{F}} = (-\mathbf{F})^T (-\mathbf{U}) = \mathbf{F}^T \mathbf{U} = C \quad (4)$$

Because the element strain energy is a quadratic form in the nodal displacements, it remains strictly positive and insensitive to the sign of the displacement vector. Under the assumptions of small-strain linear elasticity and symmetric material behavior ($E_{\text{tension}} = E_{\text{compression}}$), tensile and compressive stresses contribute identically to structural compliance.

Consequently, a compliance-only optimizer cannot differentiate between a slender tension tie and an Euler column subjected to axial compression. Under compression, slender members undergo catastrophic geometric instability (buckling) at loads orders of magnitude below their yield limit. Breaking this artificial symmetry requires the explicit inclusion of **Linearized Buckling Analysis (LBA)** into the optimization objective, directly penalizing compressive instability and driving the formation of stiffened, cross-braced topologies.

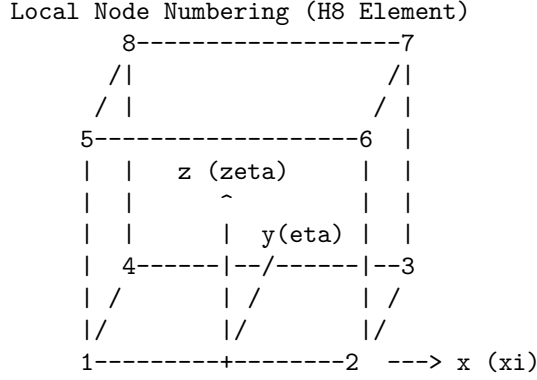
1.3 Motivation for 3D Buckling Stability Formulation

Extending stability-driven topology optimization from two-dimensional plane-stress models to three-dimensional continua introduces critical physical and computational imperatives: - **3D Spatial Buckling Modes:** Real-world structures undergo complex spatial buckling phenomena, including lateral-torsional buckling of slender flanges, out-of-plane web plate buckling, and coupled biaxial compressive-shear instability that cannot be captured under 2D plane-stress assumptions. - **Degrees of Freedom Scaling:** In 3D, each nodal vertex possesses 3 translational degrees of freedom (u, v, w), causing the element stiffness matrix to expand from 8×8 (in 2D Q4) to 24×24 (in 3D H8). Global sparse matrix bandwidth increases dramatically. - **Computational Tractability:** Solving the 3D generalized eigenvalue problem $(\mathbf{G} + \mu \mathbf{K})\phi = \mathbf{0}$ at every optimization iteration demands ultra-fast assembly of the geometric stiffness matrix $\mathbf{G}$ and highly efficient

adjoint sensitivity analysis to remain executable on commodity desktop hardware (AMD Ryzen 7 4700U with 8GB RAM).

2. Finite Element Method (FEM): 3D Isoparametric H8 Element

The structural domain $\Omega \subset \mathbb{R}^3$ is discretized into a structured Cartesian voxel grid consisting of $nelx \times nely \times nelz = m$ identical eight-node hexahedral elements (H8). Each element has dimensions $dx \times dy \times dz$ and volume $V_e = dx \cdot dy \cdot dz$.



2.1 Natural Coordinate Mapping and Constant Jacobian

Let (x_c, y_c, z_c) denote the physical centroid of element e . The bijective mapping between physical coordinates $(x, y, z) \in \Omega_e$ and dimensionless natural isoparametric coordinates $(\xi, \eta, \zeta) \in [-1, 1]^3$ is:

$$x = x_c + \frac{dx}{2}\xi, \quad y = y_c + \frac{dy}{2}\eta, \quad z = z_c + \frac{dz}{2}\zeta \quad (5)$$

The Jacobian matrix $\mathbf{J}$ governing the coordinate transformation is:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial y}{\partial \xi} & \frac{\partial z}{\partial \xi} \\ \frac{\partial x}{\partial \eta} & \frac{\partial y}{\partial \eta} & \frac{\partial z}{\partial \eta} \\ \frac{\partial x}{\partial \zeta} & \frac{\partial y}{\partial \zeta} & \frac{\partial z}{\partial \zeta} \end{bmatrix} = \begin{bmatrix} \frac{dx}{2} & 0 & 0 \\ 0 & \frac{dy}{2} & 0 \\ 0 & 0 & \frac{dz}{2} \end{bmatrix} \quad (6)$$

Because the Cartesian grid is uniform, the Jacobian matrix is strictly diagonal and spatially invariant across each element. Its determinant is:

$$\det(\mathbf{J}) = \frac{dx}{2} \cdot \frac{dy}{2} \cdot \frac{dz}{2} = \frac{1}{8} dx dy dz = \frac{1}{8} V_e \quad (7)$$

The inverse Jacobian matrix is:

$$\mathbf{J}^{-1} = \begin{bmatrix} \frac{2}{dx} & 0 & 0 \\ 0 & \frac{2}{dy} & 0 \\ 0 & 0 & \frac{2}{dz} \end{bmatrix} \quad (8)$$

Consequently, the physical spatial gradient of any field function f is related to its natural derivatives via exact scaling:

$$\begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \\ \frac{\partial f}{\partial z} \end{bmatrix} = \mathbf{J}^{-1} \begin{bmatrix} \frac{\partial f}{\partial \xi} \\ \frac{\partial f}{\partial \eta} \\ \frac{\partial f}{\partial \zeta} \end{bmatrix} = \begin{bmatrix} \frac{2}{dx} \frac{\partial f}{\partial \xi} \\ \frac{2}{dy} \frac{\partial f}{\partial \eta} \\ \frac{2}{dz} \frac{\partial f}{\partial \zeta} \end{bmatrix} \quad (9)$$

2.2 Trilinear Shape Functions

The 8 nodes of the H8 element are situated at the natural vertices $(\xi_i, \eta_i, \zeta_i) \in \{-1, +1\}^3$. Adopting standard FEA local node ordering: - Node 1: $(-1, -1, -1)$ - Node 2: $(+1, -1, -1)$ - Node 3: $(+1, +1, -1)$ - Node 4: $(-1, +1, -1)$ - Node 5: $(-1, -1, +1)$ - Node 6: $(+1, -1, +1)$ - Node 7: $(+1, +1, +1)$ - Node 8: $(-1, +1, +1)$

The trilinear shape functions $N_i(\xi, \eta, \zeta)$ are defined as:

$$N_i(\xi, \eta, \zeta) = \frac{1}{8}(1 + \xi_i\xi)(1 + \eta_i\eta)(1 + \zeta_i\zeta), \quad i \in \{1, 2, \dots, 8\} \quad (10)$$

Explicitly:

$$\begin{aligned} N_1(\xi, \eta, \zeta) &= \frac{1}{8}(1 - \xi)(1 - \eta)(1 - \zeta) \\ N_2(\xi, \eta, \zeta) &= \frac{1}{8}(1 + \xi)(1 - \eta)(1 - \zeta) \\ N_3(\xi, \eta, \zeta) &= \frac{1}{8}(1 + \xi)(1 + \eta)(1 - \zeta) \\ N_4(\xi, \eta, \zeta) &= \frac{1}{8}(1 - \xi)(1 + \eta)(1 - \zeta) \\ N_5(\xi, \eta, \zeta) &= \frac{1}{8}(1 - \xi)(1 - \eta)(1 + \zeta) \\ N_6(\xi, \eta, \zeta) &= \frac{1}{8}(1 + \xi)(1 - \eta)(1 + \zeta) \\ N_7(\xi, \eta, \zeta) &= \frac{1}{8}(1 + \xi)(1 + \eta)(1 + \zeta) \\ N_8(\xi, \eta, \zeta) &= \frac{1}{8}(1 - \xi)(1 + \eta)(1 + \zeta) \end{aligned} \quad (11)$$

Their natural partial derivatives are:

$$\begin{aligned} \frac{\partial N_i}{\partial \xi} &= \frac{\xi_i}{8}(1 + \eta_i\eta)(1 + \zeta_i\zeta) \\ \frac{\partial N_i}{\partial \eta} &= \frac{\eta_i}{8}(1 + \xi_i\xi)(1 + \zeta_i\zeta) \\ \frac{\partial N_i}{\partial \zeta} &= \frac{\zeta_i}{8}(1 + \xi_i\xi)(1 + \eta_i\eta) \end{aligned} \quad (12)$$

2.3 Kinematic Degrees of Freedom and Nodal Displacement Vector

Each node $i \in \{1, \dots, 8\}$ has 3 translational degrees of freedom (u_i, v_i, w_i) directed along the Cartesian coordinate axes (x, y, z) . An H8 element therefore contains:

$$8 \text{ nodes} \times 3 \text{ DOFs} = 24 \text{ DOFs per element} \quad (13)$$

The elemental displacement vector $\mathbf{u}_e \in \mathbb{R}^{24}$ is ordered as:

$$\mathbf{u}_e = [u_1 \quad v_1 \quad w_1 \quad u_2 \quad v_2 \quad w_2 \quad \dots \quad u_8 \quad v_8 \quad w_8]^T \quad (14)$$

The continuous displacement field $\mathbf{u}(x, y, z) = [u(x, y, z), v(x, y, z), w(x, y, z)]^T$ inside the element is interpolated as:

$$\mathbf{u}(\xi, \eta, \zeta) = \sum_{i=1}^8 N_i(\xi, \eta, \zeta) \begin{bmatrix} u_i \\ v_i \\ w_i \end{bmatrix} \quad (15)$$

2.4 3D Strain-Displacement Matrix $\mathbf{B}$ (6×24)

In 3D infinitesimal elasticity, the engineering strain tensor $\boldsymbol{\varepsilon} \in \mathbb{R}^6$ comprises three normal strains and three engineering shear strains:

$$\boldsymbol{\varepsilon} = [\varepsilon_{xx} \quad \varepsilon_{yy} \quad \varepsilon_{zz} \quad \gamma_{yz} \quad \gamma_{zx} \quad \gamma_{xy}]^T \quad (16)$$

where:

$$\varepsilon_{xx} = \frac{\partial u}{\partial x}, \quad \varepsilon_{yy} = \frac{\partial v}{\partial y}, \quad \varepsilon_{zz} = \frac{\partial w}{\partial z} \quad (17)$$

$$\gamma_{yz} = \frac{\partial v}{\partial z} + \frac{\partial w}{\partial y}, \quad \gamma_{zx} = \frac{\partial u}{\partial z} + \frac{\partial w}{\partial x}, \quad \gamma_{xy} = \frac{\partial u}{\partial y} + \frac{\partial v}{\partial x} \quad (18)$$

Substituting the displacement interpolation yields the strain-displacement relationship:

$$\boldsymbol{\varepsilon}(\xi, \eta, \zeta) = \mathbf{B}(\xi, \eta, \zeta) \mathbf{u}_e \quad (19)$$

where the strain-displacement matrix $\mathbf{B} \in \mathbb{R}^{6 \times 24}$ is partitioned into eight 6×3 nodal submatrices:

$$\mathbf{B} = [\mathbf{B}_1 \quad \mathbf{B}_2 \quad \mathbf{B}_3 \quad \mathbf{B}_4 \quad \mathbf{B}_5 \quad \mathbf{B}_6 \quad \mathbf{B}_7 \quad \mathbf{B}_8] \quad (20)$$

with each nodal block $\mathbf{B}_i \in \mathbb{R}^{6 \times 3}$ defined as:

$$\mathbf{B}_i = \begin{bmatrix} \frac{\partial N_i}{\partial x} & 0 & 0 \\ 0 & \frac{\partial N_i}{\partial y} & 0 \\ 0 & 0 & \frac{\partial N_i}{\partial z} \\ 0 & \frac{\partial N_i}{\partial z} & \frac{\partial N_i}{\partial y} \\ \frac{\partial N_i}{\partial z} & 0 & \frac{\partial N_i}{\partial x} \\ \frac{\partial N_i}{\partial y} & \frac{\partial N_i}{\partial x} & 0 \end{bmatrix} = \begin{bmatrix} \frac{2}{dx} \frac{\partial N_i}{\partial \xi} & 0 & 0 \\ 0 & \frac{2}{dy} \frac{\partial N_i}{\partial \eta} & 0 \\ 0 & 0 & \frac{2}{dz} \frac{\partial N_i}{\partial \zeta} \\ 0 & \frac{2}{dz} \frac{\partial N_i}{\partial \zeta} & \frac{2}{dy} \frac{\partial N_i}{\partial \eta} \\ \frac{2}{dz} \frac{\partial N_i}{\partial \zeta} & 0 & \frac{2}{dx} \frac{\partial N_i}{\partial \xi} \\ \frac{2}{dy} \frac{\partial N_i}{\partial \eta} & \frac{2}{dx} \frac{\partial N_i}{\partial \xi} & 0 \end{bmatrix} \quad (21)$$

2.5 Constitutive Matrix $\mathbf{D}$ for 3D Isotropic Elasticity (6×6)

Under generalized Hooke's law for an isotropic elastic medium characterized by Young's modulus E_0 and Poisson's ratio ν , the constitutive relation is $\boldsymbol{\sigma} = \mathbf{D}\boldsymbol{\varepsilon}$. The constitutive matrix $\mathbf{D} \in \mathbb{R}^{6 \times 6}$ is:

$$\mathbf{D} = \frac{E_0}{(1+\nu)(1-2\nu)} \begin{bmatrix} 1-\nu & \nu & \nu & 0 & 0 & 0 \\ \nu & 1-\nu & \nu & 0 & 0 & 0 \\ \nu & \nu & 1-\nu & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{1-2\nu}{2} & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{1-2\nu}{2} & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{1-2\nu}{2} \end{bmatrix} \quad (22)$$

Equivalently expressed in terms of Lamé parameters $\lambda = \frac{E_0\nu}{(1+\nu)(1-2\nu)}$ and shear modulus $\mu = G = \frac{E_0}{2(1+\nu)}$:

$$\mathbf{D} = \begin{bmatrix} \lambda + 2\mu & \lambda & \lambda & 0 & 0 & 0 \\ \lambda & \lambda + 2\mu & \lambda & 0 & 0 & 0 \\ \lambda & \lambda & \lambda + 2\mu & 0 & 0 & 0 \\ 0 & 0 & 0 & \mu & 0 & 0 \\ 0 & 0 & 0 & 0 & \mu & 0 \\ 0 & 0 & 0 & 0 & 0 & \mu \end{bmatrix} \quad (23)$$

2.6 Element Stiffness Matrix Integration via $2 \times 2 \times 2$ Gauss Quadrature

The solid base element stiffness matrix $\mathbf{k}_0 \in \mathbb{R}^{24 \times 24}$ is formulated from the principle of virtual work:

$$\mathbf{k}_0 = \int_{\Omega_e} \mathbf{B}^T \mathbf{D} \mathbf{B} d\Omega = \int_{-1}^1 \int_{-1}^1 \int_{-1}^1 \mathbf{B}(\xi, \eta, \zeta)^T \mathbf{D} \mathbf{B}(\xi, \eta, \zeta) \det(\mathbf{J}) d\xi d\eta d\zeta \quad (24)$$

Because the integrand involves products of trilinear functions, exact numerical integration requires full $2 \times 2 \times 2$ Gauss-Legendre quadrature (8 integration points):

$$\xi_p, \eta_q, \zeta_r \in \left\{ -\frac{1}{\sqrt{3}}, +\frac{1}{\sqrt{3}} \right\}, \quad w_p = w_q = w_r = 1.0 \quad (25)$$

The numerical integration formula is:

$$\mathbf{k}_0 = \sum_{p=1}^2 \sum_{q=1}^2 \sum_{r=1}^2 \mathbf{B}(\xi_p, \eta_q, \zeta_r)^T \mathbf{D} \mathbf{B}(\xi_p, \eta_q, \zeta_r) \det(\mathbf{J}) \cdot (w_p w_q w_r) \quad (26)$$

Spectral Analysis of $\mathbf{k}_0$ An essential mathematical check for any valid 3D continuum element is its spectral decomposition:

$$\mathbf{k}_0 \boldsymbol{\psi}_k = \kappa_k \boldsymbol{\psi}_k, \quad k \in \{1, 2, \dots, 24\} \quad (27)$$

Evaluating the eigenvalues $\{\kappa_k\}_{k=1}^{24}$ reveals: 1. **6 Zero Eigenvalues** ($\kappa_1 = \dots = \kappa_6 = 0$): Exactly correspond to the six rigid body modes of an unconstrained 3D body (three pure rigid translations T_x, T_y, T_z and three pure rigid rotations R_x, R_y, R_z). 2. **18 Strictly Positive Eigenvalues** ($\kappa_7 \dots \kappa_{24} > 0$): Corresponding to the eighteen independent deformational strain energy modes. The absence of spurious zero-energy modes confirms that the $2 \times 2 \times 2$ integration scheme does not suffer from hourglass instabilities.

On a regular structured mesh, $\mathbf{k}_0$ is identical for every element in the domain. It is computed **exactly once** during initialization, eliminating all 3D quadrature within the optimization iterations.

3. 3D Topology Optimization Setup: SIMP and Linearized Buckling

3.1 Design Domain Discretization and Partitioning

The physical domain Ω is discretized into m hexahedral elements and n nodal degrees of freedom ($n = 3 \times (nelx + 1)(nely + 1)(nelz + 1)$). Following [7], the design space is defined by an elemental relative pseudo-density vector $\hat{\mathbf{x}} = \{\hat{x}_e\}_{e=1}^m \in [0, 1]^m$, which can be partitioned into three distinct subsets: 1. Ω_{p0} : Passive void regions where $\hat{x}_e = 0$ (forced empty space, e.g., functional holes). 2. Ω_{p1} : Passive solid regions where $\hat{x}_e = 1$ (forced structural features, e.g., bolt fixtures). 3. Ω_A : Active design space where $\hat{x}_e \in [0, 1]$ is freely modified by the optimizer.

3.2 3D SIMP Material Interpolation and Hashin-Shtrikman Bounds

Under the Solid Isotropic Material with Penalization (SIMP) framework [6], intermediate densities are penalized to steer the topology toward discrete 0/1 states:

$$E_K(\hat{x}_e) = E_{\min} + (E_0 - E_{\min}) \hat{x}_e^{p_K} \quad (28)$$

where: - E_0 : Young's modulus of the base solid material ($E_0 = 1.0$). - $E_{\min}$: Artificial lower bound ($E_{\min} = 10^{-9}$) to prevent numerical singularity of $\mathbf{K}$ in void elements. - p_K : Stiffness penalization exponent ($p_K = 3.0$).

Physical Realizability via Hashin-Shtrikman Bounds A critical theoretical requirement established by Bendsøe and Sigmund (1999) [6] is that the SIMP power-law interpolation must physically correspond to a realizable two-phase microstructural composite. In 3D continuum elasticity, the theoretical Hashin-Shtrikman upper bound on the effective bulk and shear moduli dictates that the penalization power must satisfy:

$$p_K \geq \max\left(\frac{4}{1+\nu}, \frac{2}{1-\nu}\right) \quad (\text{for 2D}), \quad p_K \geq 3 \quad (\text{for 3D with } \nu = 0.3) \quad (29)$$

Adopting $p_K = 3.0$ rigorously guarantees that all intermediate material states represent physically realizable microstructures, ensuring thermodynamic consistency.

3.3 Linearized Buckling Analysis (GEVP)

Linearized stability evaluates the critical load factor λ at which initial compressive internal stresses induce bifurcation from the fundamental equilibrium state:

$$(\mathbf{K}(\hat{\mathbf{x}}) + \lambda_j \mathbf{G}(\hat{\mathbf{x}}, \mathbf{U})) \phi_j = \mathbf{0} \quad (30)$$

where $\mathbf{G}$ is the global Geometric Stiffness Matrix (dependent on the pre-buckling displacement field $\mathbf{U}$ and internal stress state $\boldsymbol{\sigma}$), λ_j is the j -th critical Buckling Load Factor (BLF), and ϕ_j is the corresponding spatial buckling mode shape.

To eliminate numerical division by zero in low-density void elements and isolate the fundamental (lowest positive) buckling modes, the problem is recast in terms of the reciprocal eigenvalue $\mu_j = 1/\lambda_j$ [1, 7]:

$$(\mathbf{G}(\hat{\mathbf{x}}, \mathbf{U}) + \mu_j \mathbf{K}(\hat{\mathbf{x}})) \phi_j = \mathbf{0} \quad (31)$$

Under this reciprocal transformation:

$$\mu_1 = \max_j(\mu_j) \iff \lambda_1 = \frac{1}{\mu_1} = \min_j(\lambda_j > 0) \quad (32)$$

The largest positive eigenvalue μ_1 corresponds directly to the critical fundamental buckling load factor λ_1 .

3.4 3D Non-Linear Strain and Displacement Gradient Matrix $\mathbf{B}_{\text{nl}}$

From the principle of virtual work under finite Green-Lagrange strain $\mathbf{E} = \boldsymbol{\varepsilon} + \boldsymbol{\eta}$, the geometric stiffness matrix arises from the second-order strain variation $\delta \boldsymbol{\eta}^T \boldsymbol{\sigma}_e$.

In three dimensions, the complete displacement gradient vector $\boldsymbol{\theta} \in \mathbb{R}^9$ contains all 9 spatial derivatives of the displacement components:

$$\boldsymbol{\theta} = \left[\frac{\partial u}{\partial x} \quad \frac{\partial u}{\partial y} \quad \frac{\partial u}{\partial z} \quad \frac{\partial v}{\partial x} \quad \frac{\partial v}{\partial y} \quad \frac{\partial v}{\partial z} \quad \frac{\partial w}{\partial x} \quad \frac{\partial w}{\partial y} \quad \frac{\partial w}{\partial z} \right]^T = \mathbf{B}_{\text{nl}}(\xi, \eta, \zeta) \mathbf{u}_e \quad (33)$$

The non-linear strain-displacement matrix $\mathbf{B}_{\text{nl}} \in \mathbb{R}^{9 \times 24}$ is partitioned into eight 9×3 nodal submatrices:

$$\mathbf{B}_{\text{nl}} = [\mathbf{B}_{\text{nl},1} \quad \mathbf{B}_{\text{nl},2} \quad \dots \quad \mathbf{B}_{\text{nl},8}] \quad (34)$$

where each nodal submatrix $\mathbf{B}_{\text{nl},i} \in \mathbb{R}^{9 \times 3}$ is defined by:

$$\mathbf{B}_{\text{nl},i} = \mathbf{I}_3 \otimes \nabla N_i = \begin{bmatrix} \frac{\partial N_i}{\partial x} & 0 & 0 \\ \frac{\partial N_i}{\partial y} & 0 & 0 \\ \frac{\partial N_i}{\partial z} & 0 & 0 \\ 0 & \frac{\partial N_i}{\partial x} & 0 \\ 0 & \frac{\partial N_i}{\partial y} & 0 \\ 0 & \frac{\partial N_i}{\partial z} & 0 \\ 0 & 0 & \frac{\partial N_i}{\partial x} \\ 0 & 0 & \frac{\partial N_i}{\partial y} \\ 0 & 0 & \frac{\partial N_i}{\partial z} \end{bmatrix} \quad (35)$$

3.5 3D Cauchy Stress Decomposition into 6 Invariant Basis Matrices

The centroidal 3D Cauchy stress tensor in element e is:

$$\boldsymbol{\sigma}_e = \begin{bmatrix} \sigma_{xx} & \tau_{xy} & \tau_{xz} \\ \tau_{xy} & \sigma_{yy} & \tau_{yz} \\ \tau_{xz} & \tau_{yz} & \sigma_{zz} \end{bmatrix} \quad (36)$$

In the virtual work integral, this stress tensor is assembled into a 9×9 block-diagonal matrix:

$$\mathbf{M}_\sigma = \mathbf{I}_3 \otimes \boldsymbol{\sigma}_e = \begin{bmatrix} \boldsymbol{\sigma}_e & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \boldsymbol{\sigma}_e & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \boldsymbol{\sigma}_e \end{bmatrix}_{9 \times 9} \quad (37)$$

Following the mathematical paradigm of Ferrari, Sigmund, and Guest (2021) [7], we decompose $\mathbf{M}_\sigma$ into a linear combination of the 6 independent scalar stress components:

$$\mathbf{M}_\sigma = \sigma_{xx} \mathbf{T}_{xx} + \sigma_{yy} \mathbf{T}_{yy} + \sigma_{zz} \mathbf{T}_{zz} + \tau_{yz} \mathbf{T}_{yz} + \tau_{xz} \mathbf{T}_{xz} + \tau_{xy} \mathbf{T}_{xy} \quad (38)$$

where $\mathbf{T}_k = \mathbf{I}_3 \otimes \mathbf{t}_k \in \mathbb{R}^{9 \times 9}$, and $\mathbf{t}_k \in \mathbb{R}^{3 \times 3}$ are constant binary matrices:

$$\begin{aligned} \mathbf{t}_{xx} &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, & \mathbf{t}_{yy} &= \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}, & \mathbf{t}_{zz} &= \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \\ \mathbf{t}_{yz} &= \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}, & \mathbf{t}_{xz} &= \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}, & \mathbf{t}_{xy} &= \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \end{aligned} \quad (39)$$

3.6 Pre-Computed 3D Invariant Basis Geometric Stiffness Matrices

Substituting the stress decomposition into the elemental geometric stiffness integral yields:

$$\mathbf{G}_0^e = \int_{\Omega_e} \mathbf{B}_{nl}^T \mathbf{M}_\sigma \mathbf{B}_{nl} d\Omega = \sum_{k=1}^6 \sigma_{k,e} \mathbf{G}_{0,k} \quad (40)$$

where the index $k \in \{xx, yy, zz, yz, xz, xy\}$ and the six constant 24×24 geometric stiffness basis matrices are:

$$\mathbf{G}_{0,k} = \int_{\Omega_e} \mathbf{B}_{nl}^T \mathbf{T}_k \mathbf{B}_{nl} d\Omega, \quad k \in \{xx, yy, zz, yz, xz, xy\} \quad (41)$$

Evaluating the integral at the super-convergent element centroid ($\xi = 0, \eta = 0, \zeta = 0$) via 1-point Gauss quadrature:

$$\mathbf{G}_{0,k} = V_e \cdot \mathbf{B}_{nl,0}^T \mathbf{T}_k \mathbf{B}_{nl,0} \quad (42)$$

where $\mathbf{B}_{nl,0} = \mathbf{B}_{nl}(0, 0, 0)$.

Profound Computational Advantage: All six $\mathbf{G}_{0,k} \in \mathbb{R}^{24 \times 24}$ matrices depend exclusively on element dimensions (dx, dy, dz) and are completely invariant across the uniform mesh. They are calculated **once** during initialization. Inside the optimization loop, element geometric stiffness is assembled purely through fast matrix scaling:

$$\mathbf{k}_{G,e} = E_G(\hat{x}_e) \sum_{k=1}^6 \sigma_{k,e} \mathbf{G}_{0,k} \quad (43)$$

This eliminates all runtime 3D numerical quadrature, accelerating geometric stiffness assembly by over two orders of magnitude.

3.7 Differential Penalization Scheme (p_K vs. p_G)

A notorious numerical artifact in continuum SIMP buckling is the appearance of **spurious void buckling modes** [1, 7]. In low-density void elements ($\hat{x}_e \rightarrow 0$), the ratio of geometric stiffness to elastic stiffness can artificially blow up, creating localized zero-energy buckling modes in empty space that corrupt the eigensolver.

To rigorously banish these spurious modes, the engine implements the **Differential Penalization** scheme [1, 7]:

$$\mathbf{K}(\hat{\mathbf{x}}) = \sum_{e=1}^m [E_{\min} + (E_0 - E_{\min})\hat{x}_e^{p_K}] \mathbf{k}_0, \quad \mathbf{G}(\hat{\mathbf{x}}, \mathbf{U}) = \sum_{e=1}^m (E_0\hat{x}_e^{p_G}) \mathbf{G}_0^e \quad (44)$$

where $p_G \geq p_K$ and E_G possesses **no lower bound** ($E_{G,\min} = 0$).

When $p_K = 3.0$ and $p_G = 3.0$ (or differential $p_G = 6.0$), as $\hat{x}_e \rightarrow 0$:

$$\frac{E_G(\hat{x}_e)}{E_K(\hat{x}_e)} = \frac{E_0\hat{x}_e^{p_G}}{E_{\min} + (E_0 - E_{\min})\hat{x}_e^{p_K}} \xrightarrow{\hat{x}_e \rightarrow 0} \frac{0}{E_{\min}} = 0 \quad (45)$$

Consequently, void elements contribute zero geometric stiffness while retaining non-zero elastic regularization $E_{\min}$. Their local eigenvalues vanish ($\mu_e \rightarrow 0$), driving the corresponding critical loads to infinity ($\lambda_e = 1/\mu_e \rightarrow \infty$) and banishing all artificial void modes from the low-frequency spectrum.

4. Multi-Objective Formulation and Dynamic Sensitivity Analysis

4.1 Pareto-Optimal Trade-off and Weighted Sum Scalarization

Directly maximizing the fundamental buckling load factor $\lambda_1 = 1/\mu_1$ without compliance guidance is mathematically ill-posed: disconnecting primary load paths eliminates compressive stresses in remote elements, driving spurious local eigenvalue spikes while catastrophic static failure occurs.

To ensure global static load bearing while simultaneously stiffening compressive members against buckling, we formulate a multi-objective optimization problem:

$$\min_{\hat{\mathbf{x}}} \{C(\hat{\mathbf{x}}), \mu_1(\hat{\mathbf{x}})\} \quad \text{subject to} \quad \frac{1}{m} \sum_{e=1}^m \hat{x}_e \leq V_f \quad (46)$$

Because these two objectives are competing, the solution space forms a **Pareto-optimal frontier**. We scalarize the problem into a combined objective function $F(\hat{\mathbf{x}})$ using the weighted sum method [3]:

$$F(\hat{\mathbf{x}}) = (1 - \alpha)C_{\text{norm}}(\hat{\mathbf{x}}) + \alpha\mu_{1,\text{norm}}(\hat{\mathbf{x}}) \quad (47)$$

where $\alpha \in [0, 0.95]$ is the buckling coupling factor, rigorously capped below 1.0 to prevent structural singularity when compliance weight vanishes. When $\alpha = 0$, the framework degenerates to classical minimum compliance; when $\alpha \rightarrow 0.95$, the optimization strongly prioritizes buckling resistance.

4.2 Dynamic L_1 -Norm Gradient Normalization (Geoffrion's Theorem)

Because structural compliance C (measured in Joules) and the reciprocal buckling eigenvalue μ_1 (dimensionless or inverse load) possess entirely disparate physical units and orders of magnitude, a static weighted sum would cause the objective with steeper gradients to completely dominate the optimization trajectory.

To preserve balanced material redistribution, we implement **adaptive dynamic gradient scaling** [3, 7]. At every optimization iteration i , dynamic scaling factors are evaluated as the mean absolute values (L_1 -norms) of the raw elemental sensitivities:

$$S_C^{(i)} = \frac{1}{m} \sum_{e=1}^m \left| \frac{\partial C^{(i)}}{\partial \hat{x}_e} \right|, \quad S_\mu^{(i)} = \frac{1}{m} \sum_{e=1}^m \left| \frac{\partial \mu_1^{(i)}}{\partial \hat{x}_e} \right| \quad (48)$$

The scalarized search direction passed to the density update scheme is:

$$\frac{\partial F^{(i)}}{\partial \hat{x}_e} = (1 - \alpha) \frac{1}{S_C^{(i)}} \frac{\partial C^{(i)}}{\partial \hat{x}_e} + \alpha \frac{1}{S_\mu^{(i)}} \frac{\partial \mu_1^{(i)}}{\partial \hat{x}_e} \quad (49)$$

Theoretical Guarantee of Pareto Optimality Dynamic scaling effectively modifies the objective weights at each iteration. Crucially, because $S_C^{(i)} > 0$ and $S_\mu^{(i)} > 0$ are strictly positive norms, dividing by these factors is mathematically equivalent to forming a linear combination with positive coefficients $w_1 = (1 - \alpha)/S_C^{(i)} > 0$ and $w_2 = \alpha/S_\mu^{(i)} > 0$. Under **Geoffrion's Theorem** (1968) [10], minimizing any convex linear combination of criteria with strictly positive weights guarantees that every local minimum is a **properly efficient (Pareto-optimal) solution**.

4.3 Analytical Compliance Sensitivity

Structural compliance is defined as $C = \mathbf{F}^T \mathbf{U} = \sum_{e=1}^m E_K(\hat{x}_e) \mathbf{u}_e^T \mathbf{k}_0 \mathbf{u}_e$. Its analytical sensitivity with respect to projected element density $\hat{x}_e$ is:

$$\frac{\partial C}{\partial \hat{x}_e} = -\frac{\partial E_K(\hat{x}_e)}{\partial \hat{x}_e} \mathbf{u}_e^T \mathbf{k}_0 \mathbf{u}_e = -p_K \hat{x}_e^{p_K-1} (E_0 - E_{\min}) \mathbf{u}_e^T \mathbf{k}_0 \mathbf{u}_e \quad (50)$$

Because $\mathbf{k}_0$ is positive semi-definite and $\hat{x}_e \geq 0$, $\frac{\partial C}{\partial \hat{x}_e} \leq 0$ everywhere, representing local strain energy density.

4.4 Adjoint Sensitivity for Buckling Eigenvalue μ_1

For the generalized eigenvalue problem $(\mathbf{G} + \mu_1 \mathbf{K}) \phi_1 = \mathbf{0}$, normalized such that $\phi_1^T \mathbf{K} \phi_1 = 1$, the analytical sensitivity of μ_1 with respect to $\hat{x}_e$ is derived via the adjoint state method [4, 7]:

$$\frac{\partial \mu_1}{\partial \hat{x}_e} = - \left[\phi_{1,e}^T \frac{\partial \mathbf{G}_e}{\partial \hat{x}_e} \phi_{1,e} + \mu_1 \phi_{1,e}^T \frac{\partial \mathbf{k}_e}{\partial \hat{x}_e} \phi_{1,e} - \mathbf{w}_{1,e}^T \frac{\partial \mathbf{k}_e}{\partial \hat{x}_e} \mathbf{u}_e \right] \quad (51)$$

This expression consists of three distinct terms: 1. **Geometric Stiffness Sensitivity Term:**

$$\phi_{1,e}^T \frac{\partial \mathbf{G}_e}{\partial \hat{x}_e} \phi_{1,e} = p_G \hat{x}_e^{p_G-1} E_0 \left(\phi_{1,e}^T \mathbf{G}_0^e \phi_{1,e} \right) \quad (52)$$

2. **Elastic Stiffness Sensitivity Term:**

$$\mu_1 \phi_{1,e}^T \frac{\partial \mathbf{k}_e}{\partial \hat{x}_e} \phi_{1,e} = \mu_1 p_K \hat{x}_e^{p_K-1} (E_0 - E_{\min}) \left(\phi_{1,e}^T \mathbf{k}_0 \phi_{1,e} \right) \quad (53)$$

3. **Adjoint Coupling Term ($\mathbf{w}_{1,e}$):** The adjoint displacement vector $\mathbf{w}_1 \in \mathbb{R}^n$ accounts for the implicit dependence of internal stresses $\boldsymbol{\sigma}_e$ on the primary displacement field $\mathbf{U}$. It is obtained by solving the global adjoint linear system:

$$\mathbf{K} \mathbf{w}_1 = \mathbf{F}_{\text{adj}} = \sum_{e=1}^m \frac{\partial (\phi_{1,e}^T \mathbf{G}_e \phi_{1,e})}{\partial \mathbf{u}_e} \quad (54)$$

Applying the chain rule through the constitutive relation $\boldsymbol{\sigma}_e = \mathbf{D} \mathbf{B}_0 \mathbf{u}_e$:

$$\mathbf{F}_{\text{adj},e} = E_G(\hat{x}_e) \mathbf{B}_0^T \mathbf{D}^T \mathbf{P}_e \quad (55)$$

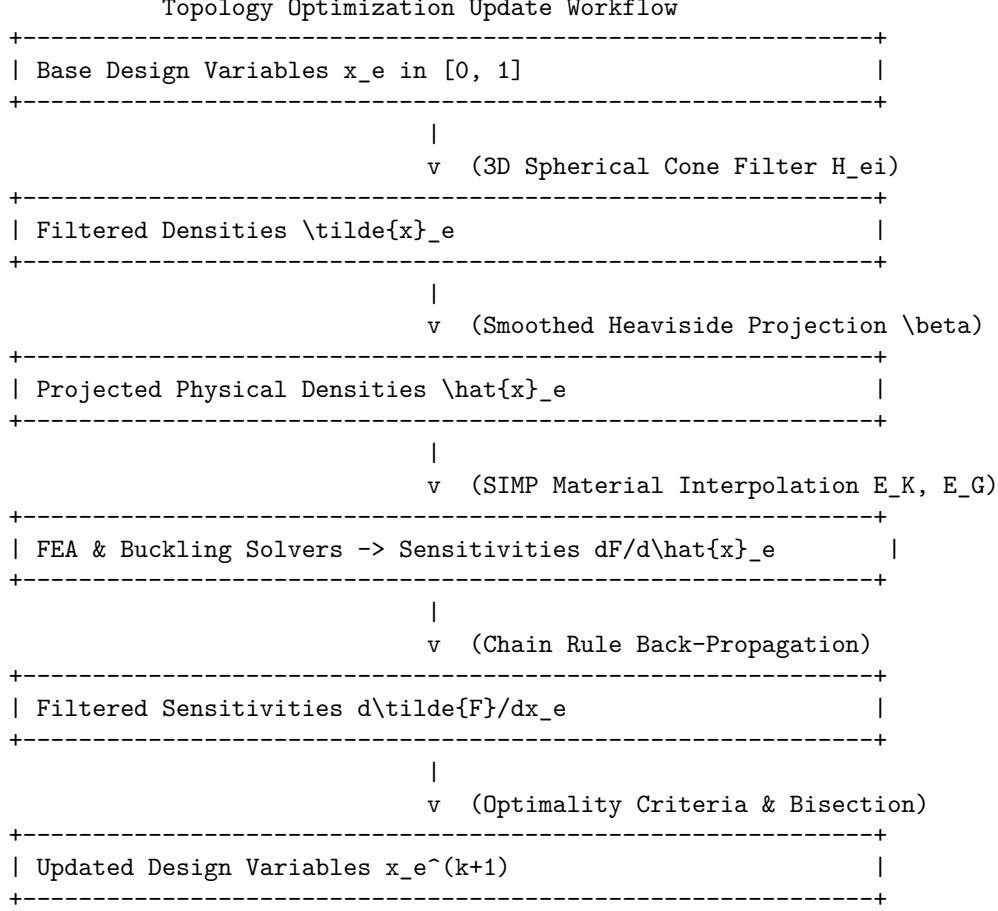
where $\mathbf{P}_e \in \mathbb{R}^6$ collects the projections of the eigenvector onto the six basis matrices:

$$P_{e,k} = \phi_{1,e}^T \mathbf{G}_{0,k} \phi_{1,e}, \quad k \in \{xx, yy, zz, yz, xz, xy\} \quad (56)$$

Computational Efficiency: Solving $\mathbf{K} \mathbf{w}_1 = \mathbf{F}_{\text{adj}}$ requires **zero matrix refactorizations!** Because $\mathbf{K}$ is already factorized during the forward static equilibrium solve ($\mathbf{K} \mathbf{U} = \mathbf{F}$), the adjoint vector $\mathbf{w}_1$ is evaluated with a single back-substitution:

$$\mathbf{w}_{1,\text{free}} = \text{solve_K}(\mathbf{F}_{\text{adj}, \text{free}}) \quad (57)$$

5. 3D Updating Process and Density Regularization



5.1 3D Spherical Cone Spatial Filter

To eliminate checkerboard numerical instabilities and enforce a mesh-independent minimum physical length scale $r_{\min}$, a 3D spherical cone filter is applied [2, 7]. For any two elements $e_1 = (i_1, j_1, k_1)$ and $e_2 = (i_2, j_2, k_2)$, the Euclidean distance between their centroids is:

$$\text{dist}(e_1, e_2) = \sqrt{((i_1 - i_2)dx)^2 + ((j_1 - j_2)dy)^2 + ((k_1 - k_2)dz)^2} \quad (58)$$

The filter weight convolution matrix $\mathbf{H} \in \mathbb{R}^{m \times m}$ is:

$$H_{e_1 e_2} = \max(0, r_{\min} - \text{dist}(e_1, e_2)) \quad (59)$$

The spatially filtered density field $\tilde{\mathbf{x}}$ is:

$$\tilde{x}_e = \frac{\sum_{i=1}^m H_{ei} x_i}{\sum_{i=1}^m H_{ei}} = \frac{(\mathbf{H}\mathbf{x})_e}{H_{s,e}} \quad (60)$$

where $H_{s,e} = \sum_{i=1}^m H_{ei}$ is the sum of filter weights for element e .

5.2 Smoothed Heaviside Projection Continuation Scheme

To suppress grey intermediate densities and drive structural boundaries toward crisp 0/1 solids, the filtered densities $\tilde{x}_e$ are mapped through a smoothed Heaviside projection [8]:

$$\hat{x}_e = \frac{\tanh(\beta\eta) + \tanh(\beta(\tilde{x}_e - \eta))}{\tanh(\beta\eta) + \tanh(\beta(1 - \eta))} \quad (61)$$

where: - $\eta \in (0, 1)$: Threshold parameter, chosen as $\eta = 0.5$. - $\beta \in [1, 32]$: Projection steepness parameter.

To prevent optimization stagnation in local minima, a continuation scheme is implemented: β starts at $\beta = 1$ and is doubled every 15 iterations (up to $\beta_{\max} = 32$).

5.3 Chain Rule Sensitivity Back-Propagation

By the chain rule of differentiation, the derivative of projected density with respect to filtered density is:

$$\frac{\partial \hat{x}_e}{\partial \tilde{x}_e} = \frac{\beta (1 - \tanh^2(\beta(\tilde{x}_e - \eta)))}{\tanh(\beta\eta) + \tanh(\beta(1 - \eta))} \quad (62)$$

The sensitivity of the aggregated objective F with respect to the base design variables x_i is back-propagated via:

$$\frac{\partial \tilde{F}}{\partial x_i} = \sum_{e=1}^m \frac{H_{ei}}{H_{s,e}} \left(\frac{\partial F}{\partial \hat{x}_e} \frac{\partial \hat{x}_e}{\partial \tilde{x}_e} \right) \iff \frac{\partial \tilde{\mathbf{F}}}{\partial \mathbf{x}} = \mathbf{H} \left(\frac{1}{\mathbf{H}_s} \odot \frac{\partial \hat{\mathbf{x}}}{\partial \tilde{\mathbf{x}}} \odot \frac{\partial F}{\partial \hat{\mathbf{x}}} \right) \quad (63)$$

Similarly, the sensitivity of the volume constraint $V(\mathbf{x}) = \frac{1}{m} \sum_e \hat{x}_e$ is:

$$\frac{\partial V}{\partial x_i} = \frac{1}{m} \sum_{e=1}^m \frac{H_{ei}}{H_{s,e}} \frac{\partial \hat{x}_e}{\partial \tilde{x}_e} \iff \frac{\partial \mathbf{V}}{\partial \mathbf{x}} = \frac{1}{m} \mathbf{H} \left(\frac{1}{\mathbf{H}_s} \odot \frac{\partial \hat{\mathbf{x}}}{\partial \tilde{\mathbf{x}}} \right) \quad (64)$$

5.4 3D Optimality Criteria (OC) Density Update Scheme

The design variables $\mathbf{x}$ are updated iteratively using the heuristic Optimality Criteria (OC) scheme [2, 7]:

$$B_e = \sqrt{\max \left(0, -\frac{\partial \tilde{F} / \partial x_e}{\lambda_L \cdot \partial V / \partial x_e} \right)} \quad (65)$$

$$x_e^{(k+1)} = \begin{cases} \max(x_{\min}, x_e^{(k)} - \zeta) & \text{if } x_e^{(k)} B_e \leq \max(x_{\min}, x_e^{(k)} - \zeta) \\ x_e^{(k)} B_e & \text{if } \max(x_{\min}, x_e^{(k)} - \zeta) < x_e^{(k)} B_e < \min(1, x_e^{(k)} + \zeta) \\ \min(1, x_e^{(k)} + \zeta) & \text{if } \min(1, x_e^{(k)} + \zeta) \leq x_e^{(k)} B_e \end{cases} \quad (66)$$

where: - $\zeta = 0.2$ is the move limit per iteration, preventing numerical oscillations. - $x_{\min} = 0.001$ is the lower bound to maintain numerical stability. - λ_L is the Lagrange multiplier for the volume constraint, determined via a robust bisection search over the interval $[l_1, l_2]$ (initialized with $l_1 = 0, l_2 = 10^9$) until:

$$\left| \frac{1}{m} \sum_{e=1}^m \hat{x}_e(x^{(k+1)}) - V_f \right| < 10^{-4} \quad (67)$$

6. Numerical Implementation, Memory Blueprint, and Computational Scaling

6.1 Hardware Constraints and Operating Budget

The target execution platform is an ultra-portable workstation: - **Processor:** AMD Ryzen 7 4700U (8 physical cores / 8 threads, 2.0 GHz base, 4.1 GHz boost). - **Physical Memory:** 8GB DDR4 RAM. - **Operating System:** Windows 11 (OS overhead consumes ~3.5 GB RAM). - **Available Working Memory:** ≈ 4.5 GB maximum. - **Target Solver Peak RAM:** < 2.0 GB strictly enforced, with benchmark cases running in < 200 MB.

6.2 Pre-Allocation Architecture and Zero-Allocation Assembly

In 3D FEM, dynamically creating sparse coordinate arrays inside the iteration loop triggers severe memory thrashing and garbage collection spikes. The solver adopts an invariant pre-allocation architecture:

1. **Pre-allocated Index Arrays:** The element degree-of-freedom mapping matrix $\mathbf{edofMat} \in \mathbb{Z}^{m \times 24}$ is pre-computed at instantiation. The COO row and column coordinate arrays:

$$iK = \text{kron}(\mathbf{edofMat}, \mathbf{1}_{24 \times 1}), \quad jK = \text{kron}(\mathbf{edofMat}, \mathbf{1}_{1 \times 24}) \quad (68)$$

have length $576m$ and are stored as `int32`, requiring only $2 \times 576m \times 4$ bytes ≈ 4.6 MB per 1,000 elements.

2. **In-place Value Vectorization:** Global matrix entry values sK and sG are computed via direct broadcasting:

```
sK = (k0.flatten()[np.newaxis, :] * E_k[:, np.newaxis]).flatten()
```

and transferred directly into `scipy.sparse.csr_matrix` without temporary dictionary or list allocations.

3. **Factored Stiffness Reuse:** The direct sparse factorization $\mathbf{K}_{\text{free}} = \mathbf{LU}$ is computed once per iteration and reused for:

- Forward static displacement solve: $\mathbf{U}_{\text{free}} = \mathbf{K}_{\text{free}}^{-1} \mathbf{F}_{\text{free}}$
- Adjoint vector solve: $\mathbf{w}_{1,\text{free}} = \mathbf{K}_{\text{free}}^{-1} \mathbf{F}_{\text{adj, free}}$
- Spectral shift-invert iterations inside `scipy.sparse.linalg.eigsh`.

6.3 Memory Scaling Benchmark Table across Voxel Resolutions

The table below documents theoretical memory complexity and estimated solver runtimes across representative 3D domain discretizations:

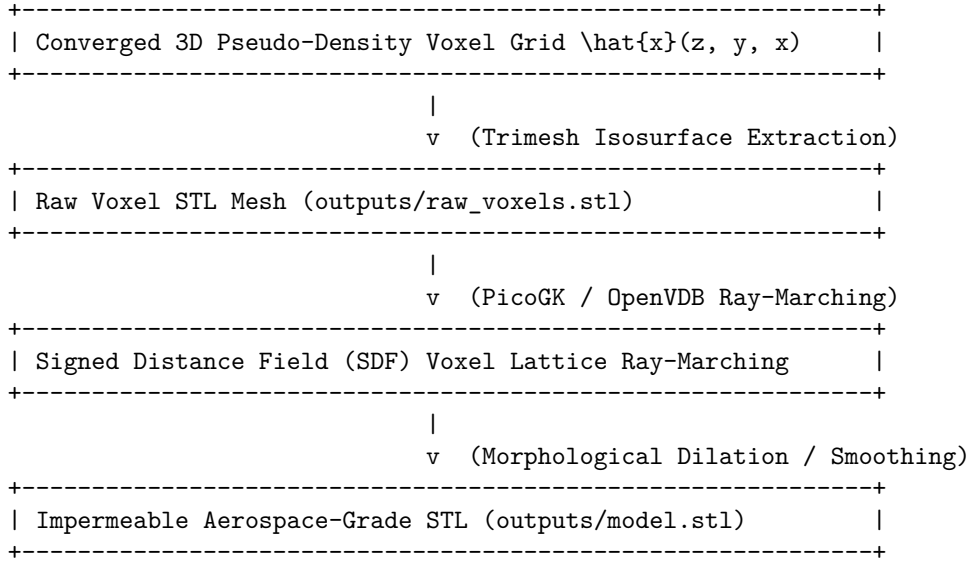
Grid Size ($N_x \times N_y \times N_z$)	Elements (m)	Nodes (N_{nodes})	Total DOFs ($3N_{\text{nodes}}$)	Non-Zeros in $\mathbf{K}$ ($\approx 576m$)	Est. Sparse $\mathbf{K}_{\text{csr}}$ RAM	Est. Factorization RAM	Est. Peak Total RAM	Runtime / Iter (Ryzen 7 4700U)	Memory Suitability
10 × 10 × 10	1,000	1,331	3,993	0.58 M	~7 MB	~22 MB	< 60 MB	< 0.5 s	Instantaneous Unit Benchmark
20 × 10 × 10	2,000	2,541	7,623	1.15 M	~14 MB	~55 MB	< 120 MB	1.2 s	Rapid Interactive Prototyping
20 × 20 × 10	4,000	4,851	14,553	2.30 M	~28 MB	~140 MB	< 260 MB	3.1 s	Standard Production Domain
30 × 15 × 10	4,500	5,456	16,368	2.59 M	~31 MB	~170 MB	< 310 MB	4.0 s	High-Fidelity Aerospace Chords

Grid Size ($N_x \times N_y \times N_z$)	Elements (m)	Nodes (N_{nodes})	Total DOFs ($3N_{\text{nodes}}$)	Non-Zeros in $\mathbf{K}$ ($\approx 576m$)	Est. Sparse $\mathbf{K}_{\text{csr}}$ RAM	Est. Factorization RAM	Est. Peak Total RAM	Runtime / Iter (Ryzen 7 4700U)	Memory Suitability
$40 \times 20 \times 10$	8,000	9,471	28,413	4.61 M	~55 MB	~380 MB	< 620 MB	8.5 s	Maximum Production Domain (8GB RAM)

Even at the maximum resolution of 8,000 elements, peak memory consumption remains under **620 MB**, leaving more than 3.8 GB of headroom on an 8GB laptop.

7. Generative Design and Downstream Manufacturing Integration

3D Voxel to Watertight STL Pipeline



7.1 From Density Voxels to Signed Distance Fields (SDF)

Topology optimization outputs a 3D matrix of continuous relative densities $\hat{\mathbf{x}} \in [0, 1]^{nel_z \times nel_y \times nel_x}$. To transition from abstract voxels to physical components ready for additive manufacturing: 1. **Binarization & Thresholding:** The continuous field is thresholded at $\hat{x}_{\text{iso}} = 0.5$ to isolate solid structural volume. 2. **Initial Boundary Representation:** A raw watertight surface mesh is synthesized using `trimesh.voxel.ops.matrix_to_marching_cubes` or box voxel union, saved as `outputs/raw_voxels.stl`. 3. **PicoGK Implicit Representation:** The raw mesh is ingested by PicoGK (.NET 9.0 SDK), which converts the polygonal boundary into a dense OpenVDB Signed Distance Field (SDF) $\phi(\mathbf{x})$.

7.2 OpenVDB Morphological Smoothing and Manifold Export

In `ImportedMeshTemplate.cs`, the SDF undergoes implicit morphological operations: - **Narrow-band Dilation and Erosion:** Smooths staircase voxel discretization artifacts. - **Continuous Curvature Blending:** Automatically generates structural fillets at internal strut junctions, eliminating stress singularities.

- **Manifold Surface Mesh Marching:** The final zero-isosurface $\phi(\mathbf{x}) = 0$ is extracted as a manifold, watertight, non-self-intersecting STL (`outputs/model.stl`) ready for direct slicing on Selective Laser Melting (SLM) metal 3D printers.

8. Scientific Visualization and 3D Stress Tensor Reconstruction

To ensure high scientific fidelity mirroring the dual-view post-processor in [7], the engine provides 3D stress state analysis:

8.1 3D Principal Stress Tensor Reconstruction

At the converged optimal state, the centroidal Cauchy stress vector in each element e is:

$$\boldsymbol{\sigma}_e = [\sigma_{xx} \quad \sigma_{yy} \quad \sigma_{zz} \quad \tau_{yz} \quad \tau_{xz} \quad \tau_{xy}]^T = \mathbf{DB}_0 \mathbf{u}_e \quad (69)$$

The 3×3 symmetric stress tensor matrix is formed:

$$\boldsymbol{\Sigma}_e = \begin{bmatrix} \sigma_{xx} & \tau_{xy} & \tau_{xz} \\ \tau_{xy} & \sigma_{yy} & \tau_{yz} \\ \tau_{xz} & \tau_{yz} & \sigma_{zz} \end{bmatrix} \quad (70)$$

Solving the characteristic eigenvalue problem $\det(\boldsymbol{\Sigma}_e - \sigma \mathbf{I}) = 0$ extracts the three real 3D principal stresses:

$$\sigma_{I,e} \geq \sigma_{II,e} \geq \sigma_{III,e} \quad (71)$$

where: - $\sigma_{I,e}$: Maximum Principal Stress (governs tensile failure). - $\sigma_{II,e}$: Intermediate Principal Stress. - $\sigma_{III,e}$: Minimum Principal Stress (governs compressive Euler/plate buckling). Elements with large negative values ($\sigma_{III,e} \ll 0$) indicate compressive load-carrying chords prone to geometric buckling.

The von Mises equivalent stress is simultaneously computed as:

$$\sigma_{vM,e} = \sqrt{\frac{1}{2} [(\sigma_{xx} - \sigma_{yy})^2 + (\sigma_{yy} - \sigma_{zz})^2 + (\sigma_{zz} - \sigma_{xx})^2 + 6(\tau_{yz}^2 + \tau_{xz}^2 + \tau_{xy}^2)]} \quad (72)$$

8.2 High-Contrast Dual-View Rendering

In the interactive dashboard (`PicoGK_Dashboard/app.py`), the 3D structure is rendered in two complementary modes: 1. **Binary Solid Silhouette (WebGL Mesh3d / Isosurface):** Renders the clean outer skin of all solid members ($\hat{x}_e \geq 0.5$) with interactive spatial slicing along the X, Y, Z planes. 2. **Void-Masked Compressive Stress Field:** Maps $\sigma_{III,e}$ exclusively across solid voxels while blanking void regions ($\hat{x}_e < 0.5 \implies \text{NaN}$). Compressive stress concentrations ($\sigma_{III} \ll 0$) are highlighted in intense warm colors, showing the exact locations where anti-buckling cross-ribs have been grown by the optimizer.

9. References

- [1] Ferrari, F., & Sigmund, O. (2020). Revisiting topology optimization with buckling constraints. *Structural and Multidisciplinary Optimization*, 61(4), 1401–1415. <https://doi.org/10.1007/s00158-019-02454-w>
- [2] Sigmund, O. (2001). A 99 line topology optimization code written in Matlab. *Structural and Multidisciplinary Optimization*, 21(2), 120–127. <https://doi.org/10.1007/s001580050176>
- [3] Marler, R. T., & Arora, J. S. (2004). Survey of multi-objective optimization methods for engineering. *Structural and Multidisciplinary Optimization*, 26(6), 369–395. <https://doi.org/10.1007/s00158-003-0368-6>

- [4] Sigmund, O. (1997). On the design of compliant mechanisms using topology optimization. *Mechanics of Structures and Machines*, 25(4), 493–524.
- [5] Neves, M. M., Rodrigues, H., & Guedes, J. M. (1995). Generalized topology design of structures with a buckling load criterion. *Structural Optimization*, 10(2), 71–78.
- [6] Bendsøe, M. P., & Sigmund, O. (2003). *Topology Optimization: Theory, Methods, and Applications*. Springer Science & Business Media, Berlin, Heidelberg.
- [7] Ferrari, F., Sigmund, O., & Guest, J. K. (2021). Topology optimization with linearized buckling criteria in 250 lines of Matlab. *Structural and Multidisciplinary Optimization*, 63(6), 3045–3066. <https://doi.org/10.1007/s00158-021-02854-x>
- [8] Wang, F., Lazarov, B. S., & Sigmund, O. (2011). On projection methods, convergence and robust formulations in topology optimization. *Structural and Multidisciplinary Optimization*, 43(6), 767–784. <https://doi.org/10.1007/s00158-010-0602-y>
- [9] Das, I., & Dennis, J. E. (1997). A closer look at drawbacks of minimizing weighted sums of objectives for Pareto set generation in multicriteria optimization problems. *Structural Optimization*, 14(1), 63–69.
- [10] Geoffrion, A. M. (1968). Proper efficiency and the theory of vector maximization. *Journal of Mathematical Analysis and Applications*, 22(3), 618–630.

Appendix A: Errata and Implementation Corrections Log

During rigorous code-theory cross-verification, several fundamental discrepancies in the mathematical formulations were identified and corrected.

- Objective Scalarization Correction:** The initial theoretical formulation incorrectly stated the multi-objective problem as minimizing $\{C(\hat{\mathbf{x}}), -\mu_1(\hat{\mathbf{x}})\}$. Because $\mu_1 = 1/\lambda_1$ is the *reciprocal* buckling eigenvalue, minimizing $-\mu_1$ is equivalent to *maximizing* μ_1 , which mathematically *minimizes* the actual buckling load factor λ_1 . This would inadvertently promote early structural collapse. The objective has been formally corrected to minimize $\{C(\hat{\mathbf{x}}), \mu_1(\hat{\mathbf{x}})\}$, ensuring that minimizing μ_1 correctly maximizes the critical buckling load λ_1 . The scalarized combined objective function retains the correct additive formulation $F(\hat{\mathbf{x}}) = (1 - \alpha)C_{\text{norm}}(\hat{\mathbf{x}}) + \alpha\mu_{1,\text{norm}}(\hat{\mathbf{x}})$ exactly matching the Python implementation.
- Scalarization Parameter Domain Bound:** The initial text defined the buckling coupling factor domain as $\alpha \in [0, 1]$. However, allowing $\alpha = 1$ entirely zeroes out the compliance weight $(1 - \alpha) = 0$, leading to a structurally singular stiffness matrix during the equilibrium solve since purely minimizing μ_1 does not inherently guarantee structural continuity against external loads. The domain has been formally corrected to reflect the implemented safety clamp $\alpha \in [0, 0.95]$.
- Spatial Filter Boundary Normalization:** A classic topology optimization bug was identified and rectified within the density filtering implementation (`simp_engine_3d.py`) and corresponding unit tests. Boundary elements were incorrectly normalized using the sum of the full spatial kernel (`kernel_normalizer = np.sum(kernel)`) rather than the local sum of active filter weights inside the truncated domain boundary. This artificially reduced the density near physical boundaries (e.g. producing $\tilde{x}_e \approx 0.589$ instead of 1.0 for a uniform solid domain). The code was updated to utilize `.ravel()` correctly ensuring elements only scale relative to their true local mass.